



PostgreSQL

What is Data?



What is Information?



Rahul Dravid scores 100 Runs in Cricket Test Match @Wankhede Stadium

What is Database?

Unstructured Data

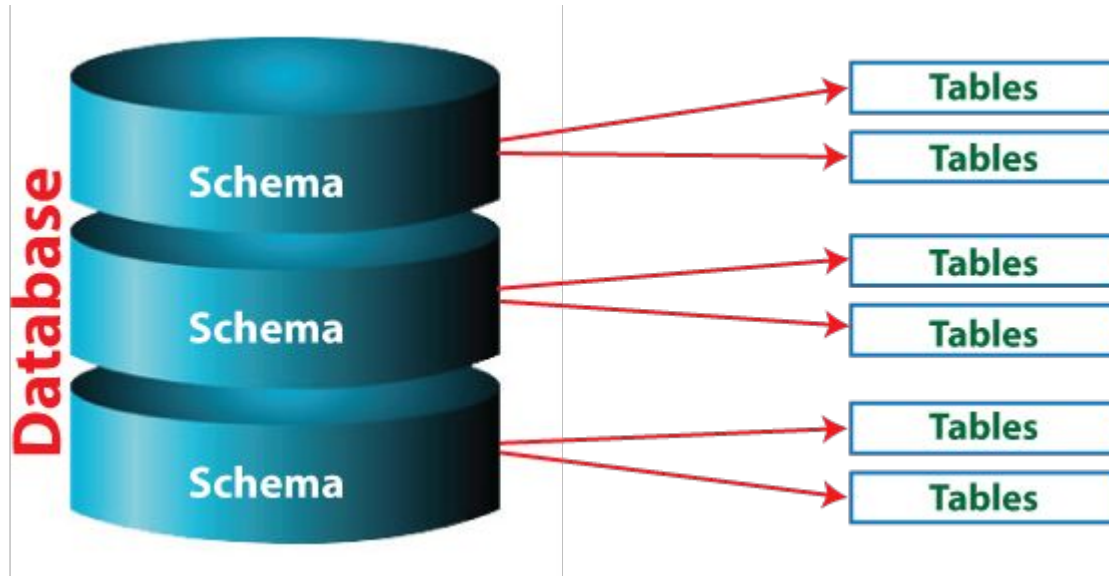


Structured Data



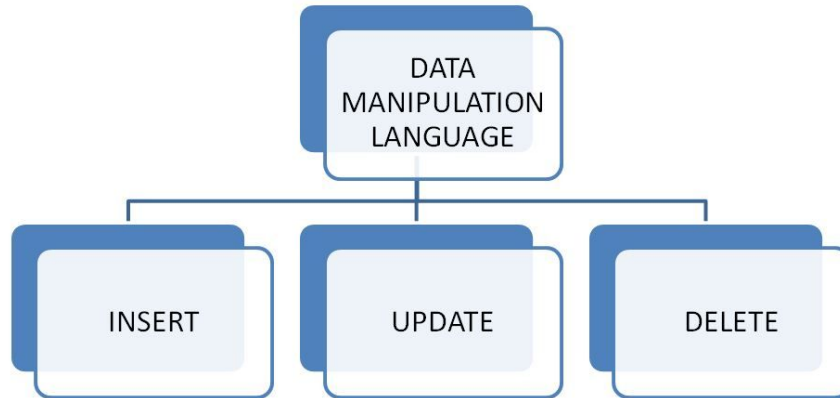
Functions of DBMS

Defining database schema



Functions of DBMS

Manipulation of the database



Rahul Dravid scores **100** Runs in Cricket Test Match @Wankhede Stadium
Sachin Tendulkar scores **150** Runs in Cricket Test Match @Wankhede Stadium

Functions of DBMS

Sharing of database



Functions of DBMS

Protection of database

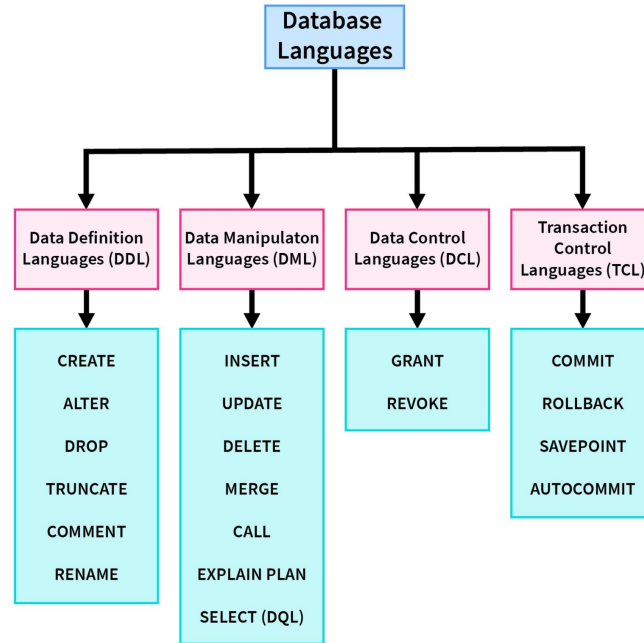


Functions of DBMS

Database recovery



Database Languages



Difference between prime and non prime attribute?

Prime attribute:

Prime attribute is the attribute which is the part of the candidate key.

Non-prime attribute:

Non -prime attribute is the attribute which is not the part of the candidate key.

Example:

Table have four attribute : A,B,C,D

candidate key : AB

then,

prime attribute is: A and B

non-prime attribute is: C and D

Various Keys in Database Management System

- Super Key
- Candidate Key
- Primary Key
- Alternate Key
- Foreign Key
- Composite Key

Super Key

- Super Key is an attribute (or a set of attributes) that uniquely identify a tuple i.e. an entity in entity set.
- It is a superset of Candidate Key, since Candidate Keys are selected from super key.

<Student>

Student_ID	Student_Enroll	Student_Name	Student_Email
S02	4545	Dave	ddd@gmail.com
S34	4541	Jack	jjj@gmail.com
S22	4555	Mark	mmm@gmail.com

Super Key

Student ID	Student Enrolment	Student Name	Student Email
S101	E201910001	Dave	dave101@gmail.com
S102	E201910002	Jack	jack102@gmail.com
S103	E201910003	Nick	nick103@gmail.com

Super Key

Following are the Super Keys for the above table

{Student_ID}

{Student_Enrolment}

{Student_Email}

{Student_ID, Student_Enrolment}

{Student_ID, Student_Name}

{Student_ID, Student_Email}

{Student_Name, Student_Enrolment}

{Student_ID, Student_Enrolment, Student_Name}

{Student_ID, Student_Enrolment, Student_Email }

{Student_ID, Student_Enrolment, Student_Name, Student_Email}

Candidate Key

A super key with no repeated attribute is called candidate key. Candidate keys are selected from the set of super keys, the only thing we take care while selecting candidate key is: It should not have any redundant attribute.

Candidate Key

Following would be the candidate key from the above

{Student_ID}

{Student_Enrolment}

{Student_Email}

Primary Key

A Primary key is selected from a set of candidate keys. There can be more than one candidate key in a relation out of which one can be chosen as primary key. For Example, `STUD_NO` as well as `STUD_PHONE` both are candidate keys for relation `STUDENT` but `STUD_NO` can be chosen as primary key (only one out of many candidate keys).

Alternate Key

The candidate key other than primary key is called as alternate key. For Example, STUD_NO as well as STUD_PHONE both are candidate keys for relation STUDENT but STUD_PHONE will be alternate key (only one out of many candidate keys).

Candidate Key

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

primary Key

Alternate Key

PostgreSQL Primary Key

A primary key is a column or a group of columns used to identify a row uniquely in a table.

You define primary keys through primary key constraints. Technically, a primary key constraint is the combination of a **not-null constraint** and a **UNIQUE constraint**.

Define primary key when creating the table

```
CREATE TABLE TABLE (  
    column_1 data_type PRIMARY KEY,  
    column_2 data_type,  
    ...  
);
```

In case the primary key consists of two or more columns, you define the primary key constraint as follows:

```
CREATE TABLE TABLE (  
    column_1 data_type,  
    column_2 data_type,  
    ...  
    PRIMARY KEY (column_1, column_2)  
);
```



PostgreSQL Primary Key

Suppose you want to add a primary key constraint to the products table, you can execute the following statement:

```
ALTER TABLE products  
ADD PRIMARY KEY (product_no);
```

PostgreSQL Primary Key

How to add an auto-incremented primary key to an existing table

Suppose, we have a vendors table that does not have any primary key.

```
CREATE TABLE vendors (name VARCHAR(255));
```

And we add few rows to the vendors table using **INSERT statement**:

```
INSERT INTO vendors (name)
```

```
VALUES
```

```
    ('Microsoft'),('IBM'),('Apple'),('Samsung');
```

PostgreSQL Primary Key

To verify the insert operation, we query data from the vendors table using the following **SELECT** statement:

```
SELECT
    *
FROM
    vendors;
```

name
Microsoft
IBM
Apple
Samsung

Now, if we want to add a primary key named id into the vendors table and the id field is auto-incremented by one, we use the following statement:

```
ALTER TABLE vendors ADD COLUMN ID SERIAL PRIMARY KEY;
```

Let's check the vendors table again.

```
SELECT
    id,name
FROM
    vendors;
```

id	name
1	Microsoft
2	IBM
3	Apple
4	Samsung

Foreign Key

A foreign key is a column which is added to create a relationship with another table. Foreign keys help us to maintain data integrity and also allows navigation between two different instances of an entity. Every relationship in the model needs to be supported by a foreign key.

PostgreSQL Foreign Key

A foreign key is a column or a group of columns in a table that reference the **primary key** of another table.

The table that contains the foreign key is called the referencing table or child table. And the table referenced by the foreign key is called the referenced table or parent table.

A table can have multiple foreign keys depending on its relationships with other tables.

PostgreSQL Foreign Key Example

The following statements create the customers and contacts tables:

```
DROP TABLE IF EXISTS customers;
```

```
DROP TABLE IF EXISTS contacts;
```

```
CREATE TABLE customers(  
    customer_id INT GENERATED ALWAYS AS IDENTITY,  
    customer_name VARCHAR(255) NOT NULL,  
    PRIMARY KEY(customer_id)  
);
```

```
CREATE TABLE contacts(  
    contact_id INT GENERATED ALWAYS AS IDENTITY,  
    customer_id INT,  
    contact_name VARCHAR(255) NOT NULL,  
    phone VARCHAR(15),  
    email VARCHAR(100),  
    PRIMARY KEY(contact_id),  
    CONSTRAINT fk_customer  
        FOREIGN KEY(customer_id)  
        REFERENCES customers(customer_id)  
);
```



PostgreSQL Foreign Key

In this example, the customers table is the parent table and the contacts table is the child table.

Each customer has zero or many contacts and each contact belongs to zero or one customer.

The customer_id column in the contacts table is the foreign key column that references the primary key column with the same name in the customers table.

The following foreign key constraint `fk_customer` in the contacts table defines the `customer_id` as the foreign key:

```
CONSTRAINT fk_customer
```

```
    FOREIGN KEY(customer_id)
```

```
        REFERENCES customers(customer_id)
```

PostgreSQL Not-Null Constraint

In database theory, NULL represents unknown or information missing. NULL is not the same as an empty string or the number zero.

To check if a value is NULL or not, you use the IS NULL boolean operator. For example, the following expression returns true if the value in the email address is NULL.

```
email_address IS NULL
```

PostgreSQL Not-Null Constraint

To control whether a column can accept NULL, you use the NOT NULL constraint:

```
CREATE TABLE table_name(  
    ...  
    column_name data_type NOT NULL,  
    ...  
);
```

If a column has a NOT NULL constraint, any attempt to **insert or update** NULL in the column will result in an error.

PostgreSQL Not-Null Constraint

Declaring NOT NULL columns

The following CREATE TABLE statement creates a new table name invoices with the not-null constraints.

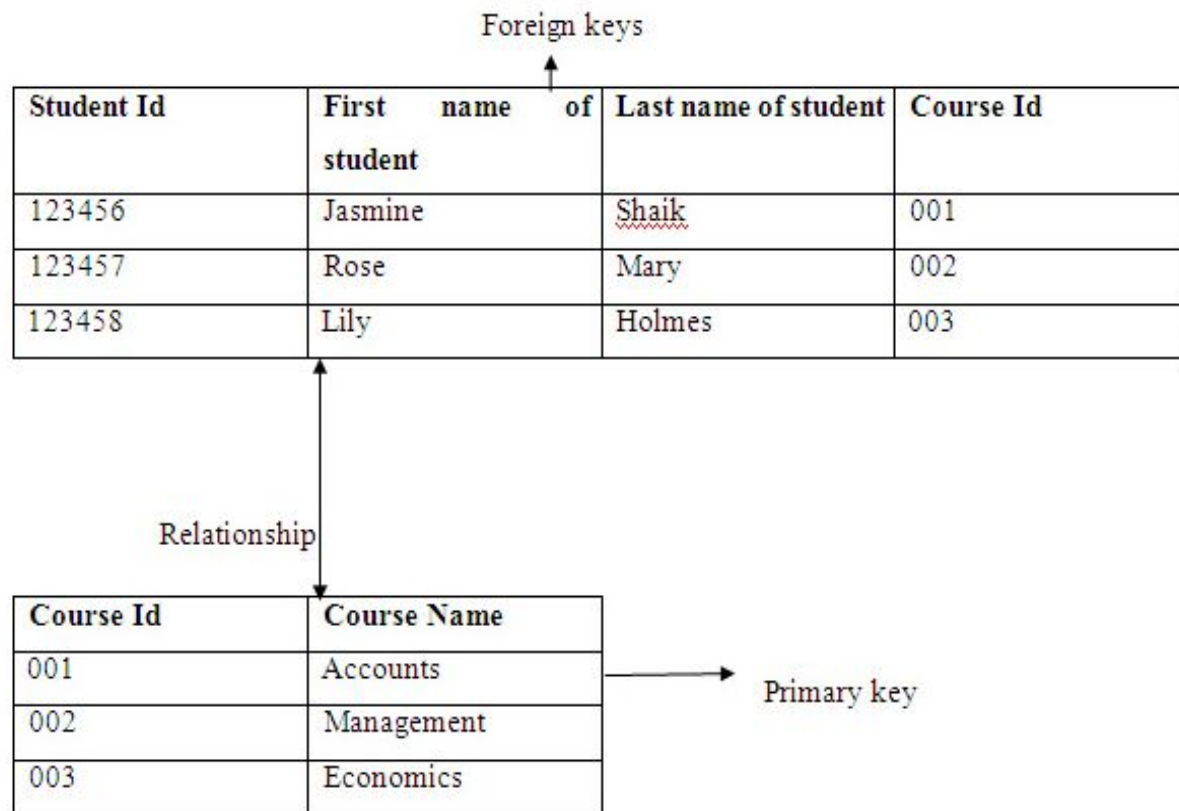
```
CREATE TABLE invoices(  
  id SERIAL PRIMARY KEY,  
  product_id INT NOT NULL,  
  qty numeric NOT NULL CHECK(qty > 0),  
  net_price numeric CHECK(net_price > 0)  
);
```

This example uses the NOT NULL keywords that follow the **data type** of the product_id and qty columns to declare NOT NULL constraints.

Why we need a Key?

- Keys help you to identify any row of data in a table. In a real-world application, a table could contain thousands of records. Moreover, the records could be duplicated. Keys ensure that you can uniquely identify a table record despite these challenges.
- Allows you to establish a relationship between and identify the relation between tables
- Help you to enforce identity and integrity in the relationship.

Foreign Key



Composite Key

A key that has more than one attributes is known as composite key. It is also known as compound key.

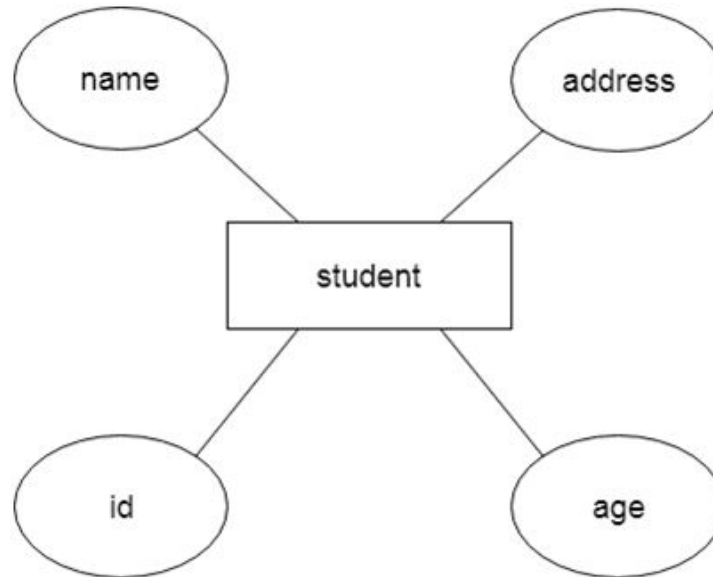
Note: Any key such as super key, primary key, candidate key etc. can be called composite key if it has more than one attributes.

E-R Model

- ER model stands for an Entity-Relationship model. It is a high-level data model. This model is used to define the data elements and relationship for a specified system.
- It develops a conceptual design for the database. It also develops a very simple and easy to design view of data.
- In ER modeling, the database structure is portrayed as a diagram called an entity-relationship diagram.

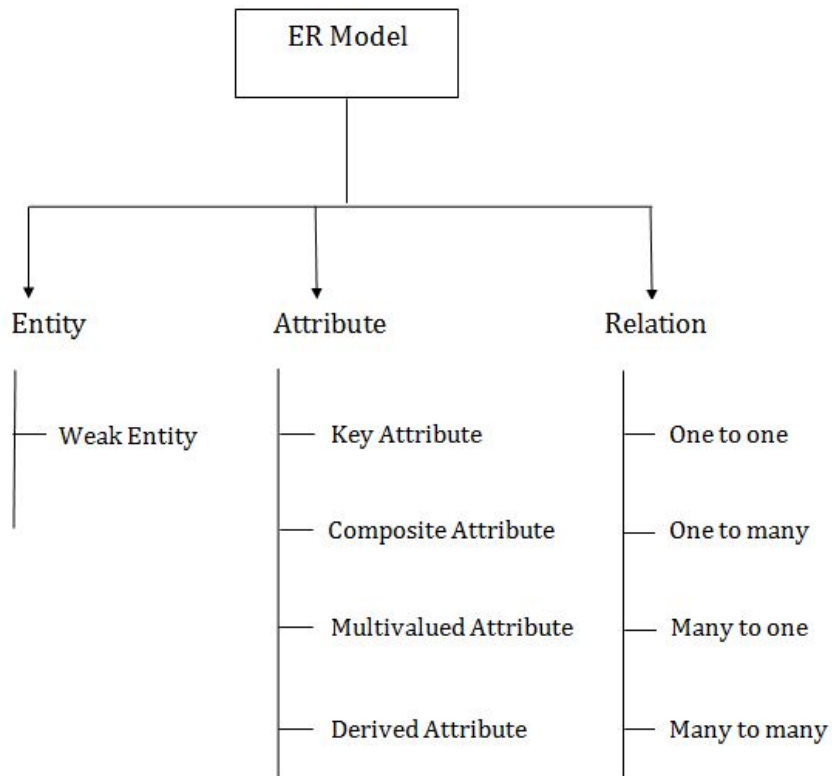
E-R Model

For example, Suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc and there will be a relationship between them.



E-R Model

Component of ER Diagram

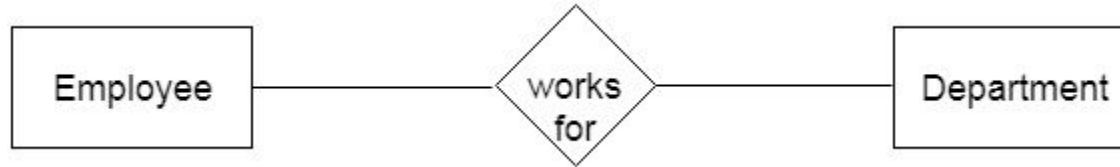


E-R Model

1. Entity:

An entity may be any object, class, person or place. In the ER diagram, an entity can be represented as rectangles.

Consider an organization as an example- manager, product, employee, department etc. can be taken as an entity.



a. Weak Entity

An entity that depends on another entity called a weak entity. The weak entity doesn't contain any key attribute of its own. The weak entity is represented by a double rectangle.

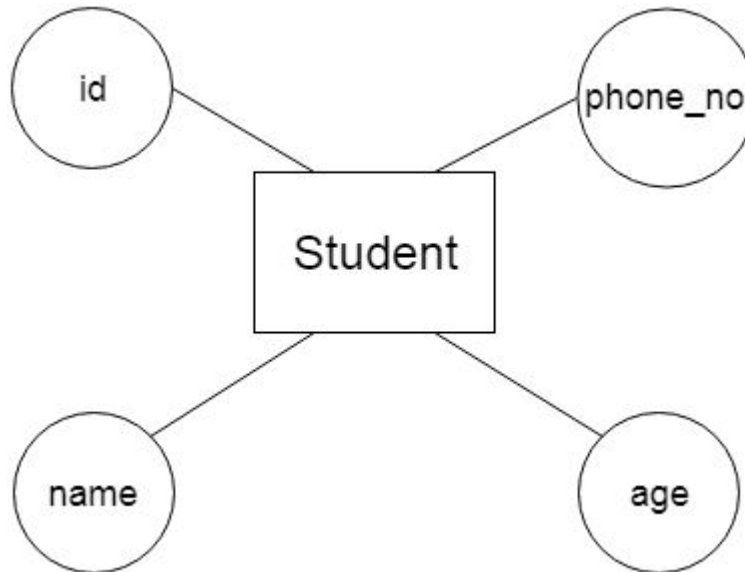


E-R Model

2. Attribute

The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute.

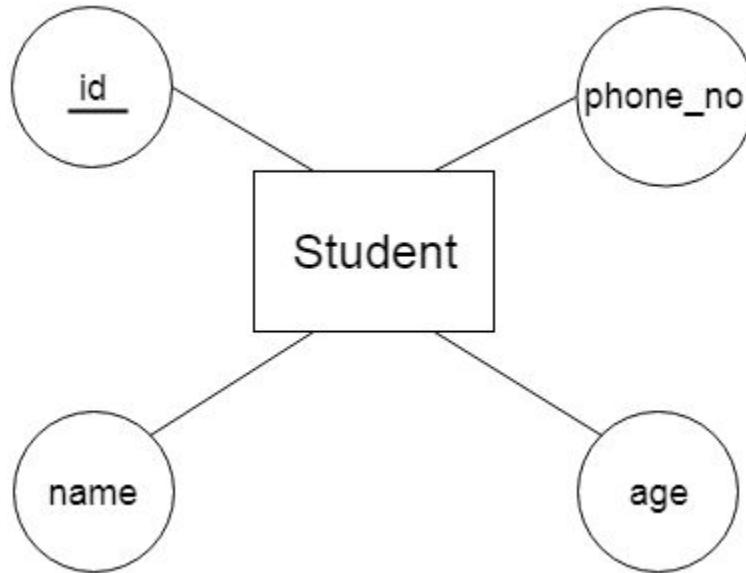
For example, id, age, contact number, name, etc. can be attributes of a student.



E-R Model

a. Key Attribute

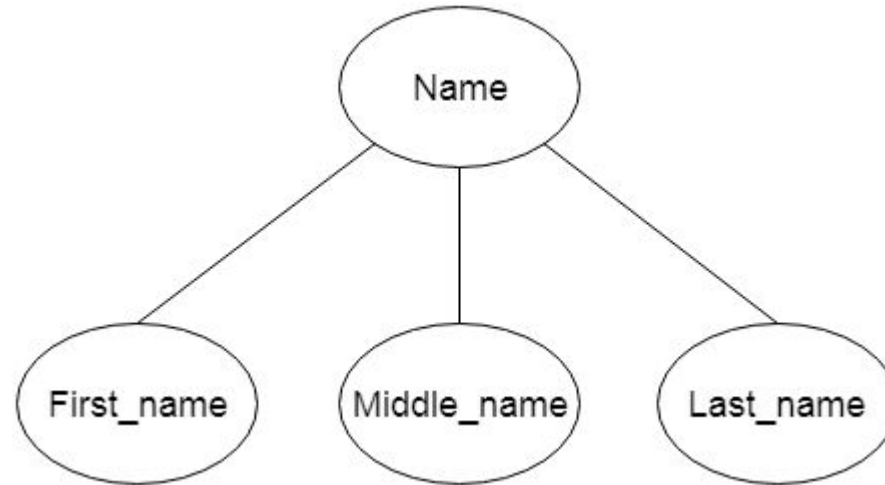
The key attribute is used to represent the main characteristics of an entity. It represents a primary key. The key attribute is represented by an ellipse with the text underlined.



E-R Model

b. Composite Attribute

An attribute that composed of many other attributes is known as a composite attribute. The composite attribute is represented by an ellipse, and those ellipses are connected with an ellipse.

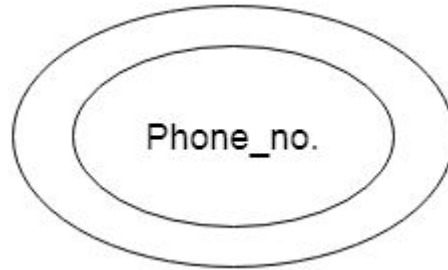


E-R Model

c. Multivalued Attribute

An attribute can have more than one value. These attributes are known as a multivalued attribute. The double oval is used to represent multivalued attribute.

For example, a student can have more than one phone number.

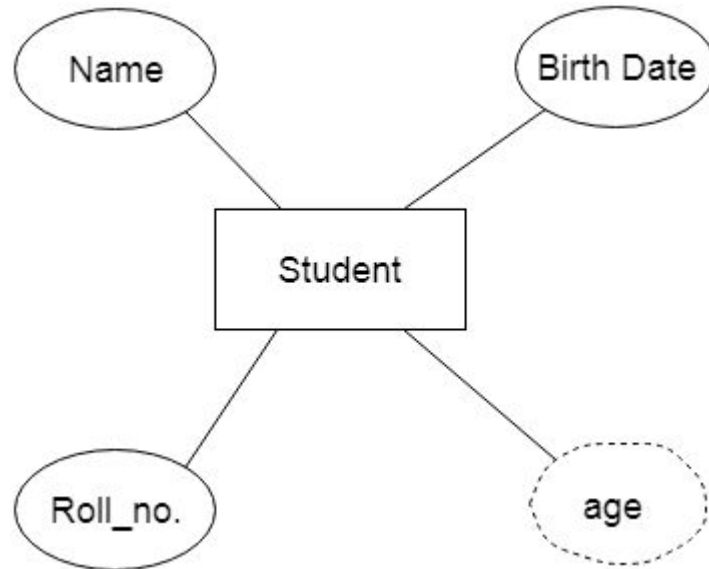


E-R Model

d. Derived Attribute

An attribute that can be derived from other attribute is known as a derived attribute. It can be represented by a dashed ellipse.

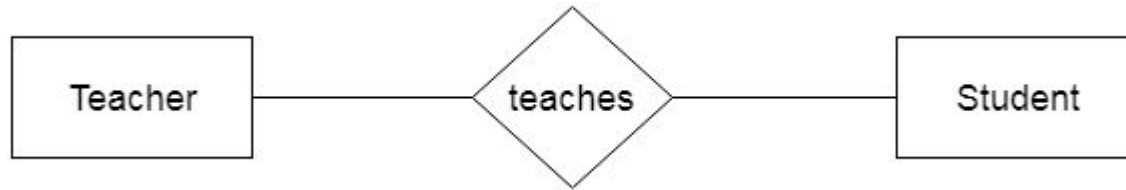
For example, A person's age changes over time and can be derived from another attribute like Date of birth.



E-R Model

3. Relationship

A relationship is used to describe the relation between entities. Diamond or rhombus is used to represent the relationship.



E-R Model

a. One-to-One Relationship

When only one instance of an entity is associated with the relationship, then it is known as one to one relationship.

For example, A female can marry to one male, and a male can marry to one female.

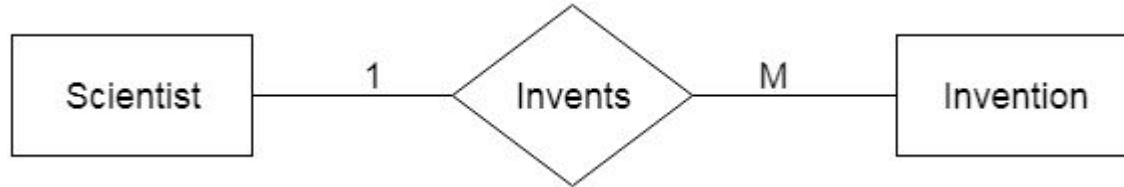


E-R Model

b. One-to-many relationship

When only one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then this is known as a one-to-many relationship.

For example, Scientist can invent many inventions, but the invention is done by the only specific scientist.

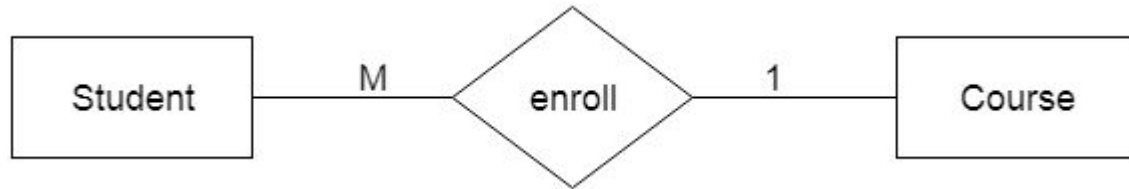


E-R Model

c. Many-to-one relationship

When more than one instance of the entity on the left, and only one instance of an entity on the right associates with the relationship then it is known as a many-to-one relationship.

For example, Student enrolls for only one course, but a course can have many students.



E-R Model

d. Many-to-many relationship

When more than one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then it is known as a many-to-many relationship.

For example, Employee can assign by many projects and project can have many employees.

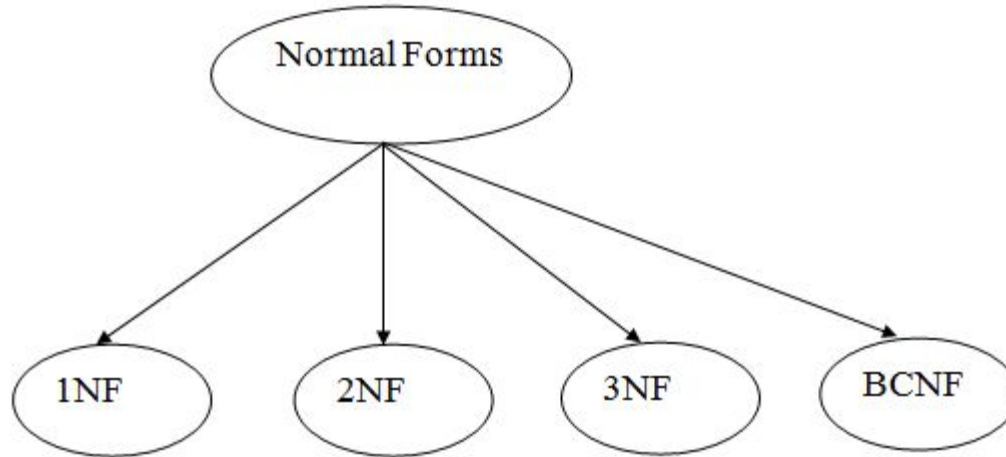


Normalization

- **Normalization** is the process of **organizing the data in the database**.
- Normalization is used to **minimize the redundancy from a relation or set of relations**. It is also used to **eliminate the undesirable characteristics like Insertion, Update and Deletion Anomalies**.
- Normalization **divides the larger table into the smaller table and links them using relationship**.
- The normal form is used to **reduce redundancy** from the database table.

Types of Normal Forms

There are the **four types** of normal forms:



Types of Normal Forms

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
4NF	A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
5NF	A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

First Normal Form (1NF)

- A relation will be 1NF if it **contains an atomic value**.
- It states that an **attribute of a table cannot hold multiple values**. It must **hold only single-valued attribute**.
- First normal form **disallows the multi-valued attribute, composite attribute, and their combinations**.

First Normal Form (1NF)

Example: Relation **EMPLOYEE** is **not in 1NF** because of **multi-valued attribute EMP_PHONE**.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385, 9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389, 8589830302	Punjab

First Normal Form (1NF)

The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385	UP
14	John	9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389	Punjab
12	Sam	8589830302	Punjab

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key

Example: Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

Second Normal Form (2NF)

TEACHER table

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

In the given table, **non-prime attribute TEACHER_AGE is dependent on TEACHER_ID** which is a **proper subset of a candidate key**. That's why it **violates the rule for 2NF**.

To **convert the given table into 2NF**, we **decompose it into two tables**:



Second Normal Form (2NF)

TEACHER_DETAIL table:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_SUBJECT table:

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Computer



Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and **not contain any transitive partial dependency**.
- 3NF is used to **reduce the data duplication**. It is also used to **achieve the data integrity**.
- If there is **no transitive dependency for non-prime attributes**, then the relation must be in third normal form.

Third Normal Form (3NF)

A relation is in third normal form if it holds at least one of the following conditions for every **non-trivial function dependency** $X \rightarrow Y$.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

Third Normal Form (3NF)

Example:

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Third Normal Form (3NF)

Super key in the table above:

`{EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on`

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.

Third Normal Form (3NF)

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal



Boyce Codd normal form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF **if every functional dependency $X \rightarrow Y$, X is the super key of the table.**
- For BCNF, **the table should be in 3NF, and for every FD, LHS is super key.**

Example: Let's assume there is a company where **employees work in more than one department.**

Boyce Codd normal form (BCNF)

EMPLOYEE table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

EMP_ID → EMP_COUNTRY

EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Boyce Codd normal form (BCNF)

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because **neither EMP_DEPT nor EMP_ID alone are keys.**

To convert the given table into BCNF, we decompose it into **three tables:**

Boyce Codd normal form (BCNF)

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India
364	UK
364	UK

EMP_DEPT

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
264	Designing
264	Testing
364	Stores
364	Developing

Boyce Codd normal form (BCNF)

Functional dependencies:

1. $\text{EMP_ID} \rightarrow \text{EMP_COUNTRY}$
2. $\text{EMP_DEPT} \rightarrow \{\text{DEPT_TYPE}, \text{EMP_DEPT_NO}\}$

Candidate keys:

For the first table: EMP_ID

For the second table: EMP_DEPT

For the third table: {EMP_ID, EMP_DEPT}

Now, this is in BCNF because **left side part of both the functional dependencies is a key.**

Fourth normal form (4NF)

- A relation will be in **4NF** if it is in **Boyce Codd normal form** and has **no multi-valued dependency**.
- For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Example

STUDENT

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

Fourth normal form (4NF)

The given **STUDENT table** is in **3NF**, but the **COURSE** and **HOBBY** are two **independent entity**. Hence, there is no relationship between COURSE and HOBBY.

In the STUDENT relation, a student with **STU_ID, 21** contains **two courses, Computer and Math** and **two hobbies, Dancing and Singing**. So **there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.**

Fourth normal form (4NF)

So to make the above table into 4NF, we can decompose it into two tables:

STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

Fifth normal form (5NF)

- A relation is in **5NF** if it is in **4NF** and not contains any **join dependency** and **joining** should be **lossless**.
- 5NF is satisfied when **all the tables are broken into as many tables as possible in order to avoid redundancy**.
- **5NF is also known as Project-join normal form (PJ/NF)**.

Fifth normal form (5NF)

Example

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

Fifth normal form (5NF)

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

P1

SEMESTER	SUBJECT
Semester 1	Computer
Semester 1	Math
Semester 1	Chemistry
Semester 2	Math

P2

SUBJECT	LECTURER
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

P3

SEMESTER	LECTURER
Semester 1	Anshika
Semester 1	John
Semester 1	John
Semester 2	Akash
Semester 1	Praveen

What is PostgreSQL?

PostgreSQL is a powerful, open source object-relational database system.

The PostgreSQL project started in 1986 at Berkeley Computer Science Department, University of California.

PostgreSQL



Install PostgreSQL

- we will show you how to **install PostgreSQL** on your local system for learning and practicing PostgreSQL.
- Version 8.0, PostgreSQL offers an installer for Windows systems that makes the installation process easier and faster. For development purpose, we will install PostgreSQL version 11.3 on Windows 10.
- There are three steps to complete the PostgreSQL installation:
 1. Download PostgreSQL installer for
 2. Install
 3. Verify the installation

Install PostgreSQL

Download PostgreSQL Installer for Windows

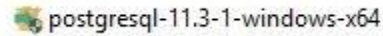
First, you need to go to the download page of [PostgreSQL installers on the EnterpriseDB](#).

Second, click the download link as shown below:

PostgreSQL Version	Linux x86-64	Linux x86-32	Mac OS X	Windows x86-64	Windows x86-32
11.3	N/A	N/A	Download	Download	N/A
10.8	Download	Download	Download	Download	Download
9.6.13	Download	Download	Download	Download	Download
9.5.17	Download	Download	Download	Download	Download
9.4.22	Download	Download	Download	Download	Download
9.3.25 (Not Supported)	Download	Download	Download	Download	Download

Install PostgreSQL step by step

Step 1. Double click on the installer file, an installation wizard will appear and guide you through multiple steps where you can choose different options that you would like to have in PostgreSQL.

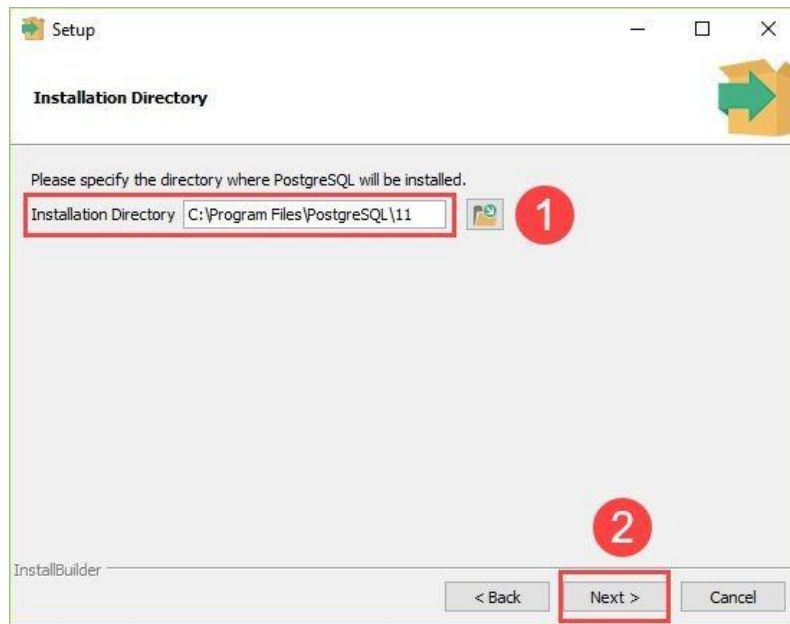


Step 2. Click the Next button



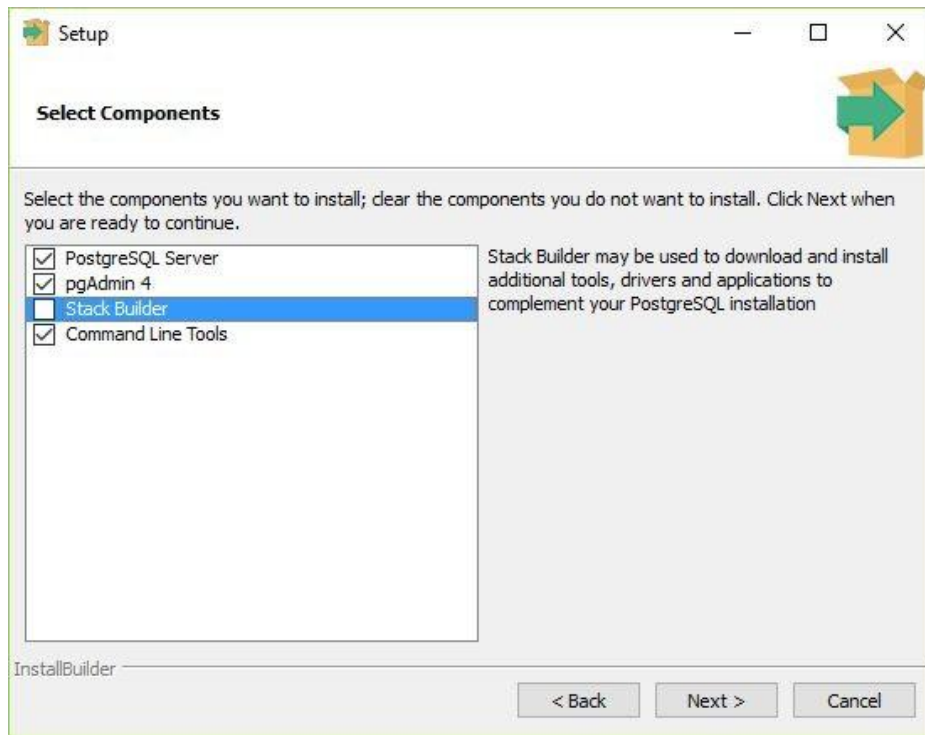
Install PostgreSQL step by step

Step 3. Specify installation folder, choose your own or keep the default folder suggested by PostgreSQL installer and click the Next button



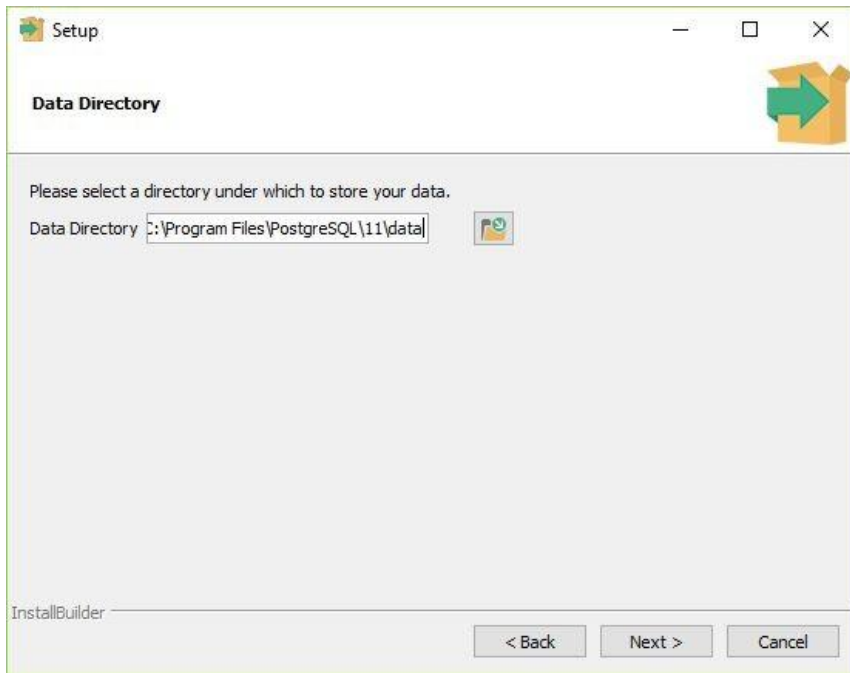
Install PostgreSQL step by step

Step 4. Select components to install and click the Next button



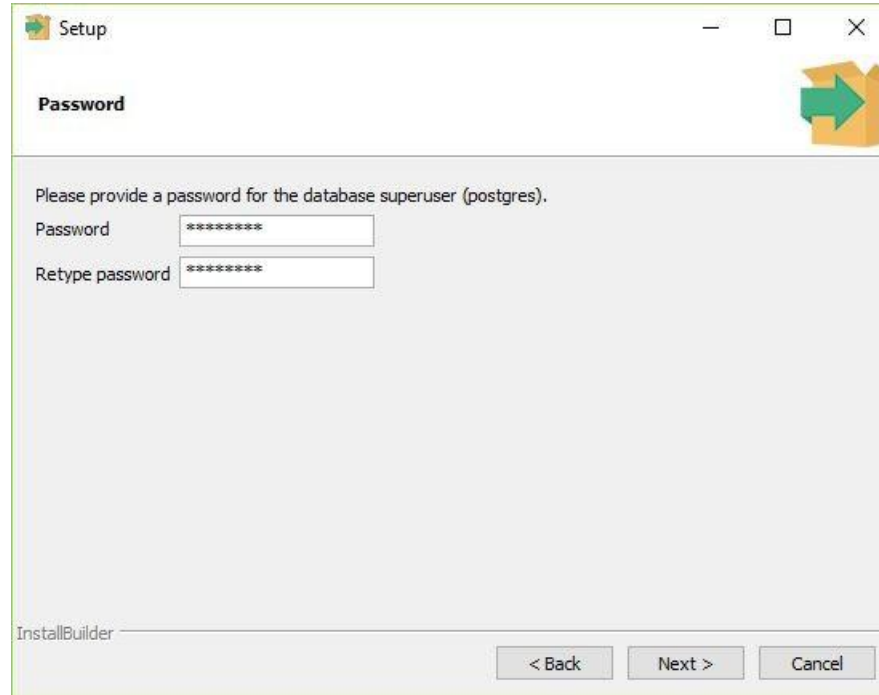
Install PostgreSQL step by step

Step 5. Select the database directory to store the data. Just leave it by default or choose your own and click the Next button.



Install PostgreSQL step by step

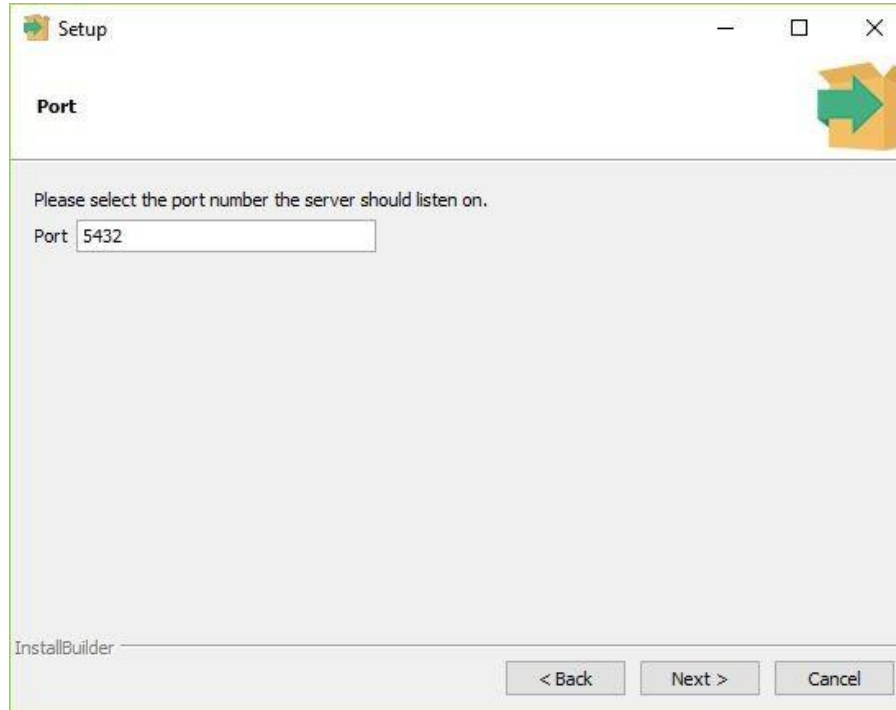
Step 6. Enter the password for the database superuser (postgres)



The screenshot shows a Windows-style window titled "Setup" with standard minimize, maximize, and close buttons. The window has a light gray background. In the top right corner, there is a large orange arrow pointing right, partially obscured by the window's border. Below the title bar, the word "Password" is displayed in bold. The main area of the window contains the text "Please provide a password for the database superuser (postgres)." followed by two input fields. The first field is labeled "Password" and the second is labeled "Retype password". Both fields contain a series of asterisks (*****). At the bottom of the window, there is a status bar that says "InstallBuilder" on the left and three buttons: "< Back", "Next >", and "Cancel" on the right.

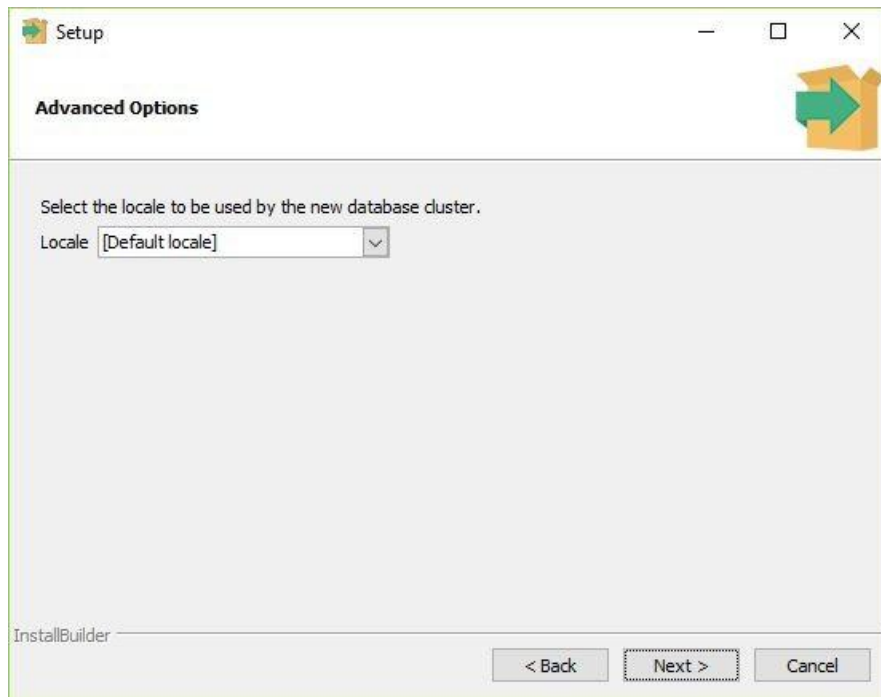
Install PostgreSQL step by step

Step 7. Enter the port for PostgreSQL. Make sure that no other applications are using this port. Leave it as default if you are unsure.



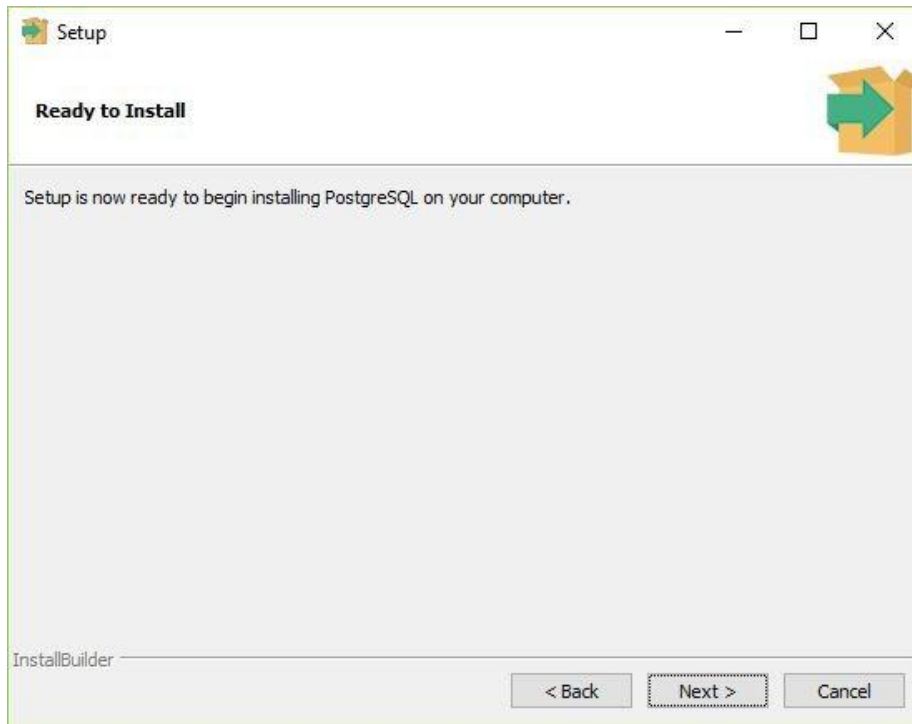
Install PostgreSQL step by step

Step 8. Choose the default locale used by the database and click the Next button.



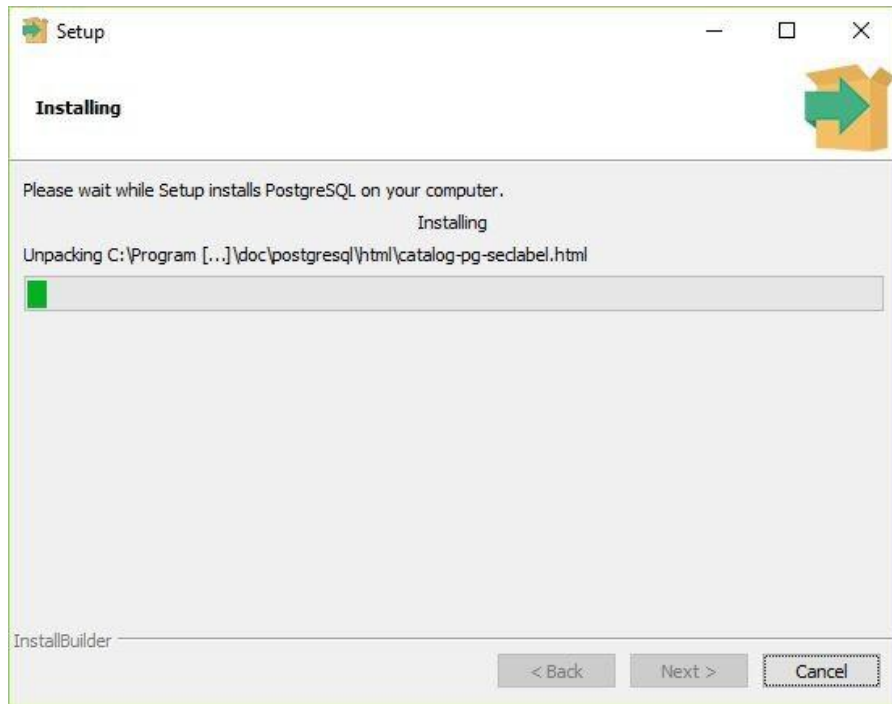
Install PostgreSQL step by step

Step 9. Ready to install PostgreSQL. Click the **Next** button to start installing.



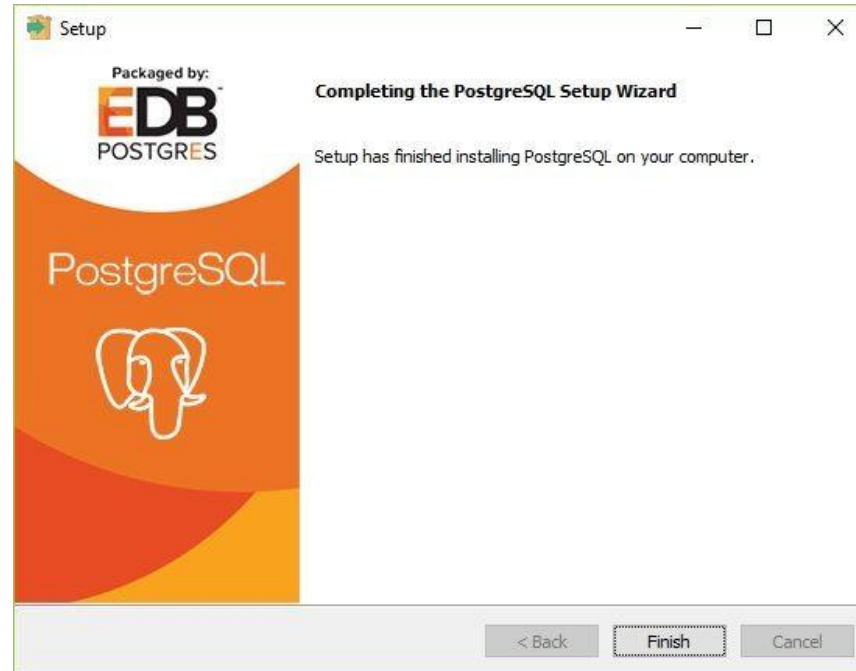
Install PostgreSQL step by step

The installation may take a few minutes to complete.



Install PostgreSQL step by step

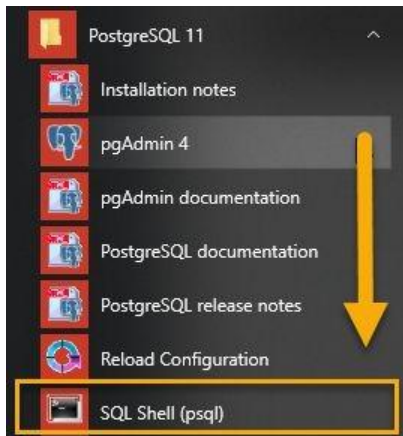
Step 10. Click the **Finish** button to complete the PostgreSQL installation.



Verify the Installation

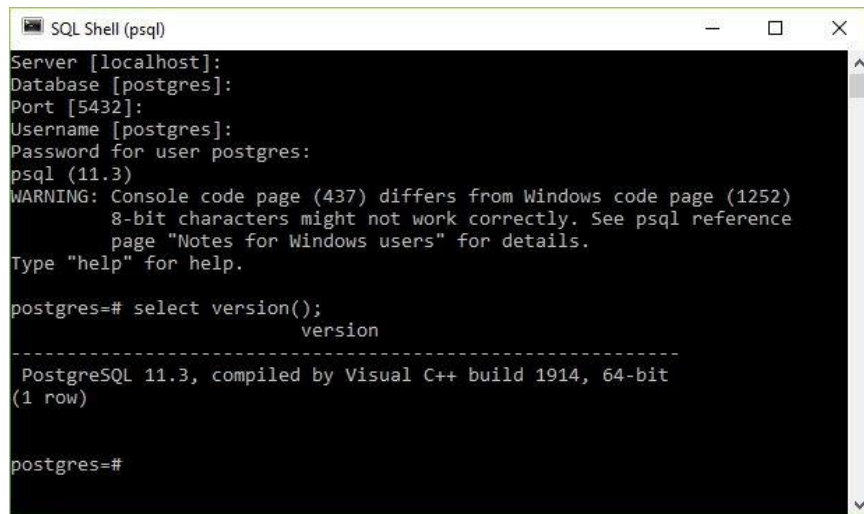
There are several ways to verify the installation. You can try to [connect to the PostgreSQL](#) database server from any client application e.g., psql and pgAdmin.

The quick way to verify the installation is through the psql program. First, click the psql icon to launch it. The psql window command line will display.



Verify the Installation

Third, issue the command `SELECT version();` you will see the result as follows:



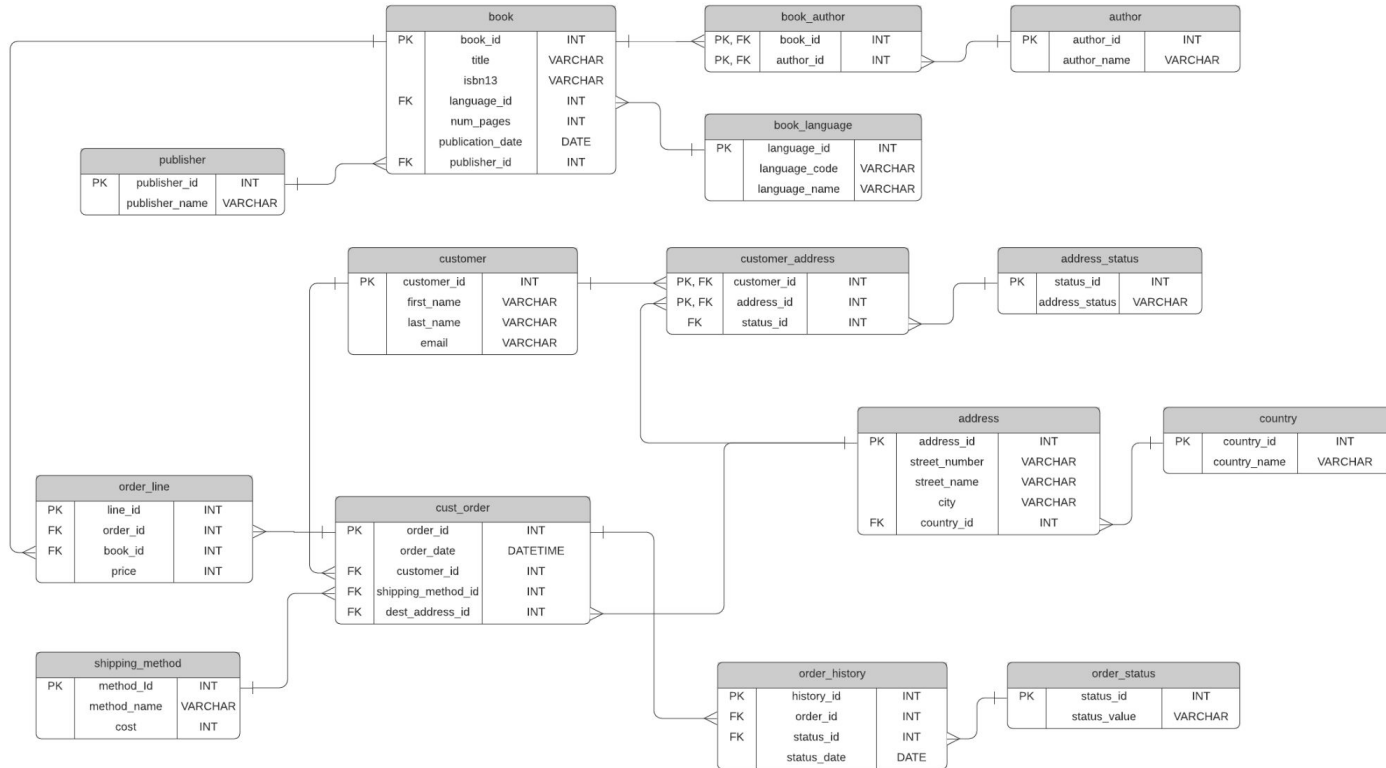
```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Password for user postgres:
psql (11.3)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

postgres=# select version();
              version
-----
PostgreSQL 11.3, compiled by Visual C++ build 1914, 64-bit
(1 row)

postgres=#
```

Congratulation! you've successfully installed PostgreSQL database server on your local system.

Sample Database-bookstore



PostgreSQL Database Query

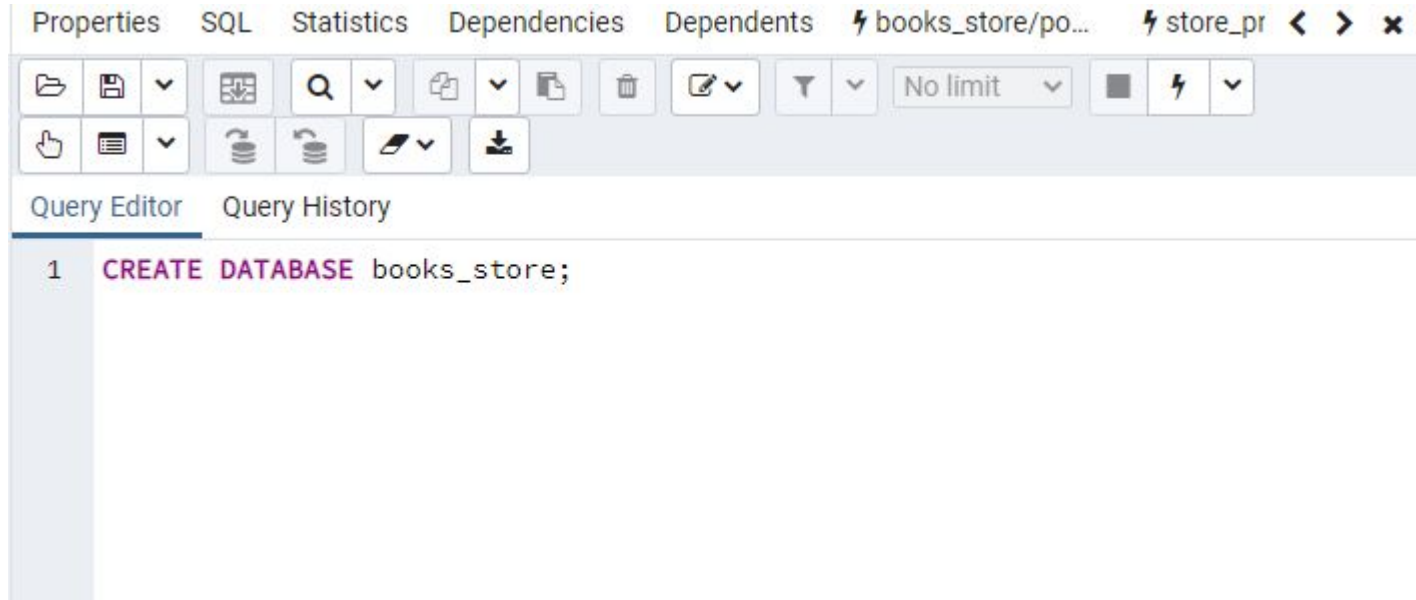
PostgreSQL Create Database

Syntax:

```
CREATE DATABASE database_name;
```



PostgreSQL Create Database using Query tool



PostgreSQL Table Clauses

PostgreSQL Create Table

In PostgreSQL, CREATE TABLE statement is used to create a new table in any of the given database

Syntax:

```
CREATE TABLE table_name(  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    .....  
    columnN datatype,  
    PRIMARY KEY( one or more  
columns )  
);
```



PostgreSQL Create Table using Query tool

See this example:

Execute the following query:

```
CREATE TABLE book (  
    book_id INT,  
    title VARCHAR(400),  
    isbn13 VARCHAR(13),  
    language_id INT,  
    num_pages INT,  
    publication_date DATE,  
    publisher_id INT,  
    CONSTRAINT pk_book PRIMARY KEY (book_id),  
);
```

Note: You can use shortcut key **Alt+Shift+Q** to open the query tool.

PostgreSQL Schema

A schema is a named collection of tables. It can also contain views, indexes, sequences, data types, operators and functions.

Syntax:

```
CREATE SCHEMA schema_name;
```

Example:

```
CREATE SCHEMA my_schema;
```



PostgreSQL INSERT Statement

Syntax:

```
INSERT INTO TABLE_NAME (column1, column2, column3,...columnN)  
VALUES (value1, value2, value3,...valueN);
```

Note: column1, column2,...columnN are the names of the columns in the table into which you want to insert data.

PostgreSQL Insert Statement Using Query Tool



The screenshot displays the PostgreSQL Query Editor interface. At the top, there is a toolbar with various icons for file operations, search, and execution. Below the toolbar, the 'Query Editor' tab is active, showing the following SQL statement:

```
1 INSERT INTO book (title, book_id, isbn13, language_id, num_pages, publication_date, publisher_id)
2 VALUES
3 ('The World's First Love: Mary Mother of God', 1, '8987059752', 2, 276, '1996-09-01', 1010),
4 ('The Illuminati', 2, '20049130001', 1, 352, '2004-10-04', 1967),
5 ('The Servant Leader', 3, '23755004321', 1, 128, '2003-03-11', 1967);
```



PostgreSQL Insert Statement Using Query Tool

Output:

Data Output		Explain	Messages	Notifications			
	book_id [PK] integer	title character varying (400)	isbn13 character varying (13)	language_id integer	num_pages integer	publication_date date	publisher_id integer
1	1	The World's First Love: Mary ...	8987059752	2	300	1996-09-01	1010
2	2	The Illuminati	20049130001	1	352	2004-10-04	1967
3	3	The Servant Leader	23755004321	1	128	2003-03-11	1967

PostgreSQL UPDATE statement

Syntax:

Following is the basic syntax of update statement:

UPDATE table_name

SET column1 = value1, column2 = value2..., columnN = valueN

WHERE [condition];

PostgreSQL UPDATE statement Example

UPDATE public.book

**SET book_id=1, num_pages=300, publication_date='1996-09-01',
publisher_id=1010**

WHERE book_id=1;

Before Update

Data Output	Explain	Messages	Notifications			
	book_id [PK] integer	title character varying (400)	isbn13 character varying (13)	language_id integer	num_pages integer	publication_date date
1	1	The World's First Love: Mary ...	8987059752	2	276	1996-09-01
2	2	The Illuminati	20049130001	1	352	2004-10-04

After Update num_pages updated whose book_id=1

	book_id [PK] integer	title character varying (400)	isbn13 character varying (13)	language_id integer	num_pages integer	publication_date date
1		1 The World's First Love: Mary ...	8987059752	2	300	1996-09-01
2		2 The Illuminati	20049130001	1	352	2004-10-04

PostgreSQL DELETE statement

Syntax :

```
DELETE FROM public.book  
WHERE <condition>;
```

Example :

```
DELETE FROM public.book  
WHERE book_id=1;
```

PostgreSQL ALTER TABLE Example

```
CREATE TABLE book (  
    book_id serial PRIMARY KEY,  
    title VARCHAR (400) NOT NULL,  
);
```

To add a new column named isbn13, you use the following statement:

```
ALTER TABLE book  
    ADD COLUMN isbn13 VARCHAR(13);
```

PostgreSQL Rename Table

To rename an existing table, you use the ALTER TABLE statement as follows:

Syntax

```
ALTER TABLE table_name  
RENAME TO new_table_name;
```

Example

```
ALTER TABLE book RENAME TO books_collection;
```

PostgreSQL SELECT



PostgreSQL SELECT

Using **SELECT** statement to query data from multiple columns example

SELECT title, publication_date, isbn13 FROM book;

	title character varying (400)	publication_date date	isbn13 character varying (13)
1	The World's First Love: Mary ...	1996-09-01	8987059752
2	The Illuminati	2004-10-04	20049130001
3	The Servant Leader	2003-03-11	23755004321
4	What Life Was Like in the Jew...	1999-09-01	34406054602
5	Cliffs Notes on Aristophanes' ...	1983-12-29	49086007763
6	Life Is a Dream and Other Spa...	2000-04-01	73999140774
7	William Goldman: Four Screen...	2000-05-01	73999254907

book		
PK	book_id	INT
	title	VARCHAR
	isbn13	VARCHAR
FK	language_id	INT
	num_pages	INT
	publication_date	DATE
FK	publisher_id	INT

PostgreSQL ORDER BY



PostgreSQL ORDER BY

Using PostgreSQL ORDER BY clause to sort rows by one column

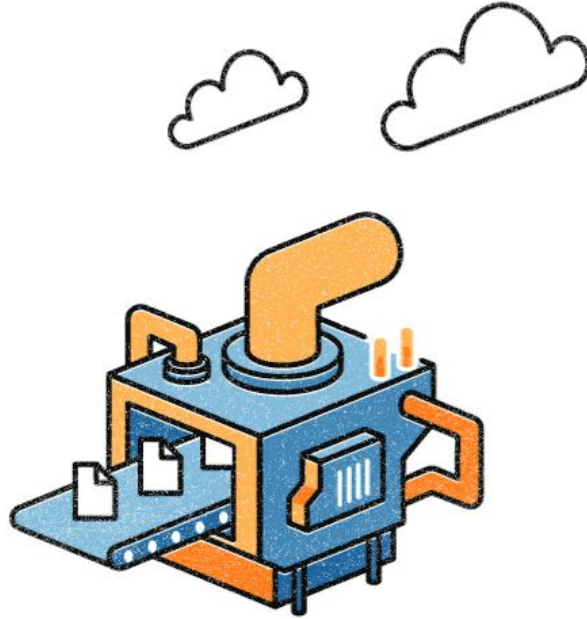
The following query use the ORDER BY clause to sort customer by the first name in ascending order:

```
SELECT  
  first_name,  
  last_name  
FROM  
  customer  
ORDER BY  
  first_name ASC;
```

	Data Output	Explain	Messages	Notifications
	 first_name character varying (200)		last_name character varying (200)	
1	Aarika		Leith-Harvey	
2	Abbot		Tesseyman	
3	Abbott		Kendrew	
4	Abby		Chevins	
5	Abel		Trower	
6	Abigael		Trowbridge	
7	Abraham		Skudder	

customer		
PK	customer_id	INT
	first_name	VARCHAR
	last_name	VARCHAR
	email	VARCHAR

Filtering Data



PostgreSQL WHERE

Using WHERE clause with the equal (=) operator example

```
SELECT  
last_name,  
first_name  
FROM customer  
WHERE  
first_name= 'Rich';
```

	last_name character varying (200)	first_name character varying (200)
1	Kirsz	Rich

customer		
PK	customer_id	INT
	first_name	VARCHAR
	last_name	VARCHAR
	email	VARCHAR

PostgreSQL WHERE using AND operator

Using WHERE clause with the AND operator example

```
SELECT
  last_name,
  first_name
FROM
  customer
WHERE
  first_name = 'Raul'
AND last_name =
  'Pentelow';
```

	last_name character varying (200)	first_name character varying (200)
1	Pentelow	Raul

customer		
PK	customer_id	INT
	first_name	VARCHAR
	last_name	VARCHAR
	email	VARCHAR

PostgreSQL WHERE using LIKE operator

Using WHERE clause with the LIKE operator example

```
SELECT  
first_name, last_name  
FROM  
customer WHERE  
first_name LIKE 'An%'
```

	first_name character varying (200)	last_name character varying (200)
1	Andrei	Jefferson
2	Anna-diana	Tilburn
3	Antonella	Scotchbourouge
4	Ann	Biggar
5	Andree	Luscott
6	Andre	Brocklehurst
7	Anthe	Erdes

customer		
PK	customer_id	INT
	first_name	VARCHAR
	last_name	VARCHAR
	email	VARCHAR

LIMIT

```
SELECT
  language_id,
  language_name
FROM
  book_language
ORDER BY
  language_id
LIMIT 4;
```

book_language		
PK	language_id	INT
	language_code	VARCHAR
	language_name	VARCHAR

	language_id [PK] integer	language_name character varying (50)
1	1	English
2	2	United States English
3	3	French
4	4	Spanish



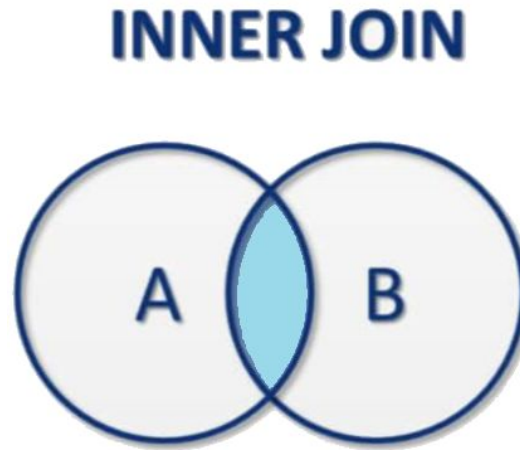
Joining Multiple Tables

INNER JOIN



INNER JOIN

The following Venn diagram illustrates the inner join:



INNER JOIN

The following statement joins the left table with the right table using the values:

```
SELECT
    book_id,title,
    publisher_id, publisher_name
FROM
    book
INNER JOIN
    publisher ON title = publisher_name;
```

book		
PK	book_id	INT
	title	VARCHAR
	isbn13	VARCHAR
FK	language_id	INT
	num_pages	INT
	publication_date	DATE
FK	publisher_id	INT

publisher		
PK	publisher_id	INT
	publisher_name	VARCHAR

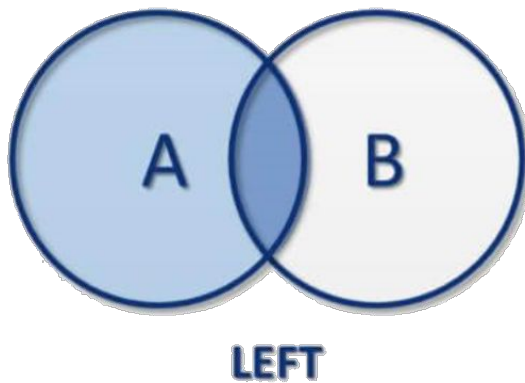
	book_id integer	title character varying (400)	publisher_id integer	publisher_name character varying (400)
1	152	Island	1035	Island
2	1320	Pan	1436	Pan
3	5763	Everyman	650	Everyman
4	6356	Poppy	1580	Poppy

LEFT JOIN



LEFT JOIN

The following Venn diagram illustrates the left join:



LEFT JOIN

PostgreSQL left join

The following statement joins the left table with the right table using **left join (or left outer join)**:

```
SELECT
    book_id,title,
    publisher_id, publisher_name
FROM
    book
LEFT JOIN
    publisher ON title = publisher_name;
```

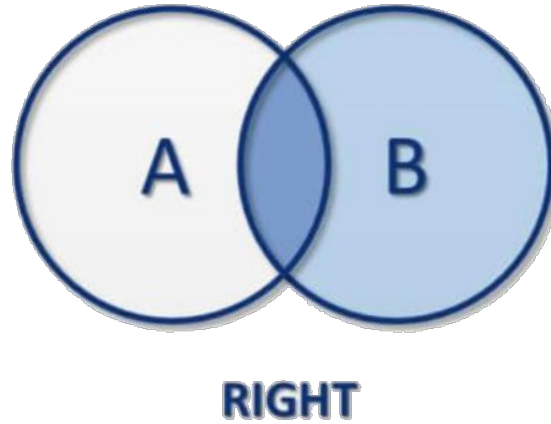
	book_id integer	title character varying (400)	publisher_id integer	publisher_name character varying (400)
149	149	The Modern Mind: An Intellect...	[null]	[null]
150	150	Sharpe's Prey (Sharpe #5)	[null]	[null]
151	151	She Went All the Way	[null]	[null]
152	152	Island	1035	Island
153	153	Planet of the Apes: The Fall	[null]	[null]
154	154	Planet of the Apes: Colony	[null]	[null]
155	155	Last Chance Saloon	[null]	[null]

book		
PK	book_id	INT
	title	VARCHAR
	isbn13	VARCHAR
FK	language_id	INT
	num_pages	INT
	publication_date	DATE
FK	publisher_id	INT

publisher		
PK	publisher_id	INT
	publisher_name	VARCHAR

RIGHT JOIN

The following Venn diagram illustrates the right join:



RIGHT JOIN

```
SELECT
    a.book_id,a.title,
    b.publisher_id, b.publisher_name
FROM
    book a
RIGHT JOIN
    publisher b
ON a.title = b.publisher_name;
```

	book_id integer	title character varying (400)	publisher_id integer	publisher_name character varying (400)
1	152	Island	1035	Island
2	1320	Pan	1436	Pan
3	5763	Everyman	650	Everyman
4	6356	Poppy	1580	Poppy
5	[null]	[null]	2196	Wiley (TP)
6	[null]	[null]	782	Gollancz
7	[null]	[null]	2107	Viking Press

book		
PK	book_id	INT
	title	VARCHAR
	isbn13	VARCHAR
FK	language_id	INT
	num_pages	INT
	publication_date	DATE
FK	publisher_id	INT

publisher		
PK	publisher_id	INT
	publisher_name	VARCHAR

Functions - Aggregate Functions (Count, Sum, Max, Min, Avg)

COUNT Function

The PostgreSQL **COUNT** function counts a number of rows or non-NULL values against a specific column from a table. When an **asterisk(*)** is used with count function the total number of rows returns.

Syntax:

```
COUNT (* | [DISTINCT] ALL | column_name)
```

book		
PK	book_id	INT
	title	VARCHAR
	isbn13	VARCHAR
FK	language_id	INT
	num_pages	INT
	publication_date	DATE
FK	publisher_id	INT

COUNT Function with GROUP BY

`SELECT title ,COUNT(*) FROM book GROUP By title;`

`SELECT COUNT(*) FROM book WHERE title='Victoria';`

`SELECT COUNT(ALL title ) FROM book WHERE title='Victoria';` all count

`SELECT COUNT(DISTINCT title ) FROM book WHERE title='Victoria';` Unique value

`SELECT COUNT(DISTINCT customer_name) from customers;`

`SELECT COUNT(ALL customer_name) from customers;`



	title	count
	character varying (400)	bigint
20	For the Relief of Unbearable U...	1
21	The Art of Loving	2
22	Victoria	1
23	Maison Ikkoku Volume 9 (Mai...	1
24	Welcome to the Great Mysteri...	1
25	You Can't Go Home Again	1
26	The Discomfort Zone: A Perso...	1
27	Selected Philosophical Writings	1
28	Civilization and Its Discontents	2

book		
PK	book_id	INT
	title	VARCHAR
	isbn13	VARCHAR
FK	language_id	INT
	num_pages	INT
	publication_date	DATE
FK	publisher_id	INT

SUM Function

SUM function

The SUM function in PostgreSQL returns the sum of values for the list of selected columns.

Syntax

```
SUM (* | [DISTINCT] ALL | column_name)
```

SUM Function Example

PostgreSQL SUM()

```
SELECT SUM(cost) FROM shipping_method;
```



	sum	
	numeric	
1		51.20

shipping_method		
PK	method_Id	INT
	method_name	VARCHAR
	cost	INT

MAX Function

The PostgreSQL MAX function returns the maximum value, specified by expression in a set of aggregated rows.

Syntax:

```
MAX (* | [DISTINCT] ALL | column_name)
```

MAX Function Example

PostgreSQL MAX() function

```
SELECT MAX(cost) FROM shipping_method;
```



	max	
	numeric	🔒
1	24.50	

shipping_method		
PK	method_Id	INT
	method_name	VARCHAR
	cost	INT

MIN Function

The MIN function determines the smallest of all selected values of a column.

Syntax:

```
MIN (* | [DISTINCT] ALL | column_name)
```

MIN Function Example

PostgreSQL MIN()

```
SELECT MIN(cost) FROM shipping_method;
```

	min	
	numeric	
1		5.90

shipping_method		
PK	method_Id	INT
	method_name	VARCHAR
	cost	INT

AVG Function

The **AVG** function determines the average of all selected values of a column.

Syntax:

```
AVG (* | [DISTINCT] ALL | column_name)
```

AVG Function Example

PostgreSQL AVG() function

```
SELECT AVG(cost) FROM shipping_method;
```



	avg	
	numeric	
1	12.80000000	

shipping_method		
PK	method_Id	INT
	method_name	VARCHAR
	cost	INT

Stored Procedure

A **Stored Procedure** is a precompiled set of SQL statements that can be executed on demand. It can contain logic like loops, conditionals, and variable handling, allowing developers to encapsulate business logic in the database. Stored procedures are often used to perform complex operations in a single call, improving performance and maintainability.

Stored procedures

The following illustrates the basic **syntax of the create procedure statement**:

```
create [or replace] procedure procedure_name(parameter_list)
language plpgsql
as $$
declare
-- variable declaration
begin
-- stored procedure body
end; $$
```

Stored procedures

In this syntax:

- First, specify the name of the stored procedure after the create procedure keywords.
- Second, define parameters for the stored procedure. A stored procedure can accept zero or more parameters.
- Third, specify plpgsql as the procedural language for the stored procedure. Note that you can use other procedural languages for the stored procedure such as SQL, C, etc.
- Finally, use the dollar-quoted string constant syntax to define the body of the stored procedure.

PostgreSQL CREATE PROCEDURE statement examples

We will use the following **accounts table** for the demonstration:

```
drop table if exists accounts;
```

```
create table accounts (  
    id int generated by default as identity,  
    name varchar(100) not null,  
    balance dec(15,2) not null,  
    primary key(id)  
);
```

```
insert into accounts(name,balance)  
values('Bob',10000);
```

```
insert into accounts(name,balance)  
values('Alice',10000);
```

PostgreSQL CREATE PROCEDURE statement examples

The following statement **shows the data from the accounts table:**

```
select * from accounts;
```

	id integer	name character varying (100)	balance numeric (15,2)
1	1	Bob	10000.00
2	2	Alice	10000.00

PostgreSQL CREATE PROCEDURE statement examples

The following example creates a stored procedure named transfer that transfers a specified amount of money from one account to another.

```
create or replace procedure transfer(  
    sender int,  
    receiver int,  
    amount decimal  
)  
language plpgsql  
as $$  
begin  
    -- subtracting the amount from the sender's account  
    update accounts  
    set balance = balance - amount  
    where id = sender;  
  
    -- adding the amount to the receiver's account  
    update accounts  
    set balance = balance + amount  
    where id = receiver;  
    commit;  
end;$$
```

PostgreSQL CREATE PROCEDURE statement examples

Calling a stored procedure

To call a stored procedure, you use the CALL statement as follows:

```
call stored_procedure_name(argument_list);
```

For example, this statement invokes the transfer stored procedure to transfer \$1,000 from Bob's account to Alice's account.

```
call transfer(1,2,1000);
```

The following statement verifies the data in the accounts table after the transfer:

```
SELECT * FROM accounts;
```

	id integer	name character varying (100)	balance numeric (15,2)
1	1	Bob	9000.00
2	2	Alice	11000.00

It worked as expected.

Stored procedures

Summary

- Use create procedure statement to define a new stored procedure.
- Use the call statement to invoke a stored procedure.

Triggers

A **Trigger** in SQL is a database object that is automatically executed or fired when specific events (like INSERT, UPDATE, or DELETE) occur on a table. Triggers allow you to define custom behavior to respond to these events.

PostgreSQL Triggers

A PostgreSQL trigger is a **function** invoked automatically whenever an event associated with a table occurs. An event could be any of the following: **INSERT, UPDATE, DELETE or TRUNCATE**.

A trigger is a special **user-defined function** associated with a table. To create a new trigger, you define a trigger function first, and then bind this trigger function to a table. The difference between a trigger and a user-defined function is that a **trigger is automatically invoked when a triggering event occurs**.

PostgreSQL Create Triggers

To create a new trigger in PostgreSQL, you follow these steps:

- First, create a trigger function using **CREATE FUNCTION** statement.
- Second, bind the trigger function to a table by using **CREATE TRIGGER** statement.

Create trigger function syntax

```
CREATE FUNCTION trigger_function()  
  RETURNS TRIGGER  
  LANGUAGE PLPGSQL  
  AS $$  
  BEGIN  
    -- trigger logic  
  END;  
  $$
```

Once you define a trigger function, you can bind it to one or more trigger events such as **INSERT, UPDATE, and DELETE**.

PostgreSQL Create Triggers

Introduction to PostgreSQL CREATE TRIGGER statement

The **CREATE TRIGGER** statement creates a new trigger. The following illustrates the basic **syntax of the CREATE TRIGGER statement**:

```
CREATE TRIGGER trigger_name
  {BEFORE | AFTER} { event }
  ON table_name
  [FOR [EACH] { ROW | STATEMENT }]
  EXECUTE PROCEDURE trigger_function
```

PostgreSQL Create Triggers Example

The following statement create a new table called **employees**:

```
DROP TABLE IF EXISTS employees;
```

```
CREATE TABLE employees(  
    id INT GENERATED ALWAYS AS IDENTITY,  
    first_name VARCHAR(40) NOT NULL,  
    last_name VARCHAR(40) NOT NULL,  
    PRIMARY KEY(id)  
);
```

Suppose that when the name of an employee changes, you want to log the changes in a separate table called **employee_audits** :

```
CREATE TABLE employee_audits (  
    id INT GENERATED ALWAYS AS IDENTITY,  
    employee_id INT NOT NULL,  
    last_name VARCHAR(40) NOT NULL,  
    changed_on TIMESTAMP(6) NOT NULL  
);
```

PostgreSQL Create Triggers Example

First, create a new function called log_last_name_changes:

```
CREATE OR REPLACE FUNCTION log_last_name_changes()  
  RETURNS TRIGGER  
  LANGUAGE PLPGSQL  
  AS  
  $$  
  BEGIN  
    IF NEW.last_name <> OLD.last_name THEN  
      INSERT INTO employee_audits(employee_id,last_name,changed_on)  
        VALUES(OLD.id,OLD.last_name,now());  
    END IF;  
  
    RETURN NEW;  
  END;  
  $$
```

PostgreSQL Create Triggers Example

Second, **bind the trigger function to the employees table**. The **trigger name** is **last_name_changes**. Before the value of the **last_name** column is updated, the trigger function is automatically **invoked to log the changes**.

```
CREATE TRIGGER last_name_changes
  BEFORE UPDATE
  ON employees
  FOR EACH ROW
  EXECUTE PROCEDURE log_last_name_changes();
OR
EXECUTE FUNCTION log_last_name_changes();
```

Note:

For PostgreSQL 11 and later, you should use **EXECUTE FUNCTION**.

For PostgreSQL versions before 11, you must use **EXECUTE PROCEDURE**.

PostgreSQL Create Triggers Example

Third, **insert** some rows into the **employees** table:

```
INSERT INTO employees (first_name, last_name)
VALUES ('John', 'Doe');
```

```
INSERT INTO employees (first_name, last_name)
VALUES ('Lily', 'Bush');
```

Fourth, examine the contents of the **employees** table:

```
SELECT * FROM employees;
```

	id integer	first_name character varying (40)	last_name character varying (40)
1	1	John	Doe
2	2	Lily	Bush

PostgreSQL Create Triggers Example

Suppose that Lily Bush changes her last name to Lily Brown.

Fifth, update Lily's last name to the new one:

```
UPDATE employees
```

```
SET last_name = 'Brown'
```

```
WHERE ID = 2;
```

Sixth, check if the last name of Lily has been updated:

```
SELECT * FROM employees;
```

	id integer	first_name character varying (40)	last_name character varying (40)
1	1	John	Doe
2	2	Lily	Brown

As you can see from the output, Lily's last name has been updated.

PostgreSQL Create Triggers Example

Seventh, verify the contents of the **employee_audits** table:

```
SELECT * FROM employee_audits;
```

	id integer	employee_id integer	last_name character varying (40)	changed_on timestamp without time zone
1	1	2	Bush	2020-07-30 17:29:04.248925

The change was logged in the employee_audits table by the trigger.

In this tutorial, you have learned how to use the **PostgreSQL CREATE TRIGGER** to create a new trigger.

PostgreSQL Views

PostgreSQL Views

Introduction

A view is a data object which does not contain any data. Contents of the view are the resultant of a base table. They are operated just like a base table but they don't contain any data of their own. The difference between a view and a table is that views are definitions built on top of other tables (or views). If data is changed in the underlying table, the same change is reflected in the view. A view can be built on top of a single or multiple tables.

How to Create PostgreSQL View?

CREATE VIEW defines a view of a query. The view is not physically materialized. Instead, the query is run every time the view is referenced in a query.

Syntax:

```
CREATE [ OR REPLACE ] [ TEMP | TEMPORARY ] [ RECURSIVE ] VIEW name [ ( column_name [,  
...] ) ]  
    [ WITH ( view_option_name [= view_option_value] [, ... ] ) ]  
AS query
```

CREATE OR REPLACE VIEW is similar, but if a **view of the same name already exists**, it is **replaced**. If a schema name is given (for example, CREATE VIEW hr.myview ...) then the view is created in the specified schema. Otherwise, it is created in the current schema. Temporary views exist in a special schema, so a schema name cannot be given when creating a temporary view. The name of the view must be distinct from the name of any other view, table, sequence, index or foreign table in the same schema.

PostgreSQL CREATE VIEW

```
CREATE VIEW customer_view  
AS SELECT customer_id, first_name, last_name  
FROM customer;
```

```
SELECT * FROM customer_view;
```

	customer_id integer	first_name character varying (200)	last_name character varying (200)
1	1	Ursola	Purdy
2	2	Ruthanne	Vatini
3	3	Reidar	Turbitt
4	4	Rich	Kirsch
5	5	Carline	Kupis

customer		
PK	customer_id	INT
	first_name	VARCHAR
	last_name	VARCHAR
	email	VARCHAR

The background of the slide is a solid blue color. Overlaid on this background are several light blue, semi-transparent rectangles of various sizes and orientations, creating a layered, geometric effect. The rectangles are positioned in the corners and along the sides, leaving the center area clear for the text.

Thank You